

Документация проекта Positioning

Проект positioning - Python-прототип пассивной акустической локализации БПЛА по разности времен прихода сигнала на четыре направленных микрофона. Он нужен для практической части: воспроизводимо сгенерировать или прочитать сигнал, подготовить окна, проверить задержки, получить направление/координаты, сгладить результат во времени и сохранить протоколы.

Назначение

Практическая часть должна показать не только математический решатель, но и рабочий программный конвейер:

```
конфигурация -> сигнал -> окна -> предобработка -> TDOA -> проверка задержек -> локализация
-> трекинг -> JSON/CSV/ПМИ
```

Текущая реализация закрывает структуру проекта, модели данных, JSON-конфигурацию, синтетический и файловый вход, предобработку, оценку TDOA через GCC-PHAT, робастный TDOA-решатель, локализацию, сопровождение, журналирование и автоматизированный протокол ПМИ.

Структура каталогов

```
positioning/
  configs/
    default.json
  docs/
    ALGORITHM_TDOA_DETAILED.md
    MODEL_STATUS.md
    PMI_PROTOCOL_SUMMARY.md
    PROJECT_OVERVIEW.md
    USER_GUIDE.md
  legacy_cpp/
    main.cpp
    validator.cpp
    validator_using_main.cpp
  src/
    positioning/
      cli.py
      config.py
      environment.py
      geometry.py
      gcc_phat.py
      localization.py
      models.py
      offline_io.py
      output.py
      pmi.py
      preprocess.py
      simulation.py
      tdoa.py
      tracking.py
      validation.py
  tests/
  tools/
    demo.py
    export_docs_to_pdf.py
    validate.py
  pyproject.toml
  README.md
```

src/positioning является единственным активным исходным кодом. legacy_cpp хранит старый C++-вариант только для сравнения и не участвует в запуске.

Основные модули

geometry.py содержит базовые 3D-операции: вектор, скалярное и векторное произведение, норму, расстояние, центрост и решение малой системы 3x3.

`environment.py` рассчитывает скорость звука по температуре и влажности.

`models.py` содержит формальные модели:

- `MicrophoneArray` - четыре микрофона, центр, проверка геометрии, нормаль плоскости;
- `Environment` - параметры среды и скорость звука;
- `AudioFrame` - четырехканальные аудиоданные;
- `PreprocessDiagnostics` - энергия, корреляция и информативность окна;
- `PairDelay` - одна TDOA-пара;
- `TdoaEstimate` - набор TDOA-пар;
- `LocalizationResult` - структурированный результат локализации;
- `AudioConfig`, `LocalizationConfig`, `AppConfig` - конфигурация проекта.

`config.py` загружает и валидирует JSON-конфигурацию.

`simulation.py` генерирует синтетический 4-канальный сигнал по заданной точке источника и сохраняет метаданные сценария. Также модуль умеет генерировать движущийся источник: серия импульсов записывается в один 4-канальный файл, а в `metadata` сохраняется массив `trajectory`.

`offline_io.py` читает и пишет 4-канальные CSV/WAV-файлы и JSON-метаданные.

`preprocess.py` разбивает сигнал на окна, удаляет DC, нормализует каналы, применяет Ханн-окно и частотную маску, рассчитывает диагностику информативности.

`gcc_phat.py` оценивает задержки между парами каналов по алгоритму GCC-PHAT и формирует `TdoaEstimate` для локализатора.

`tdoa.py` проверяет физическую допустимость и транзитивность задержек.

`localization.py` реализует режимы `direction` и `position`, робастную отбраковку ложных пар и расчет доверия.

`tracking.py` сглаживает поток результатов по окнам, отбраковывает резкие низкодостоверные скачки и оценивает скорость изменения направления.

`output.py` сохраняет JSON-результат и CSV-журнал.

`pmi.py` выполняет набор проверок по ПМИ и формирует Markdown/JSON/CSV-протоколы.

`cli.py` объединяет все в команды командной строки.

`tools/export_docs_to_pdf.py` экспортирует README и основные Markdown-документы из `docs` в PDF. PDF складываются в `docs/pdf`, включая общий файл `positioning_documentation.pdf`.

`visualization.py` открывает демонстрационное Tkinter-окно для сравнения исходной и распознанной траектории в режиме потоковой обработки.

Конфигурация

Базовый файл `configs/default.json`:

```
{
  "array": {
    "microphones": [
      {"x": 0.0, "y": 0.0, "z": 0.0},
      {"x": 0.2, "y": 0.0, "z": 0.0},
      {"x": 0.2, "y": 0.2, "z": 0.0},
      {"x": 0.0, "y": 0.2, "z": 0.0}
    ]
  },
  "audio": {
    "sample_rate": 48000,
    "window_size": 2048,
```

```

    "overlap": 0.5,
    "frequency_band": [100.0, 5000.0]
  },
  "environment": {
    "temperature_c": 20.0,
    "humidity_percent": 0.0
  },
  "localization": {
    "mode": "position"
  }
}

```

Проверяются следующие ошибки:

- файл не найден или JSON синтаксически неверен;
- не хватает обязательных секций;
- задано не четыре микрофона;
- микрофоны вырождены или не дают рабочую плоскость;
- частота дискретизации, размер окна или частотная полоса некорректны;
- overlap не лежит в допустимом диапазоне;
- температура/влажность дают некорректную среду;
- режим локализации не равен direction или position.

Конвейер работы

1. Демонстрация

Команда `positioning demo` строит синтетические времена прихода от фиксированной цели и передает их в решатель. Это быстрый способ проверить, что конфигурация, скорость звука и локализация работают.

2. Синтетический сигнал

Команда `positioning generate-synthetic` создает 4-канальный CSV или WAV. Источник задается координатами, частотой, амплитудой и временем излучения. Дополнительно включаются шум, выброс и отраженный путь.

Рядом сохраняется файл метаданных. Он нужен для воспроизводимости: в нем указаны параметры сценария, координаты цели и расчетные времена прихода.

3. Предобработка

Команда `positioning preprocess` читает CSV/WAV, разбивает сигнал на окна, обрабатывает каждое окно и сохраняет обработанные окна в отдельные CSV-файлы. В `preprocess.meta.json` попадает список окон и диагностика качества.

Визуализация привязана к этапу локализации: команда `positioning localization`

`--show-visualization` открывает окно с микрофонной решеткой, исходной точкой или траекторией из `metadata` и распознанными оценками по окнам. `preprocess` не считает координаты и не открывает графические окна. Если `metadata` содержит только одну точку `target`, визуализация ограничивается первым окном и выводит предупреждение. Это защищает демонстрацию от ситуации, когда шумовые оценки по пустым окнам выглядят как ложная траектория.

4. Локализация

Локализация работает с `TdoaEstimate`. В файловом режиме этот объект формируется из предобработанных аудиоокон через GCC-PHAT. В синтетических тестах он также может формироваться напрямую из известных времен прихода.

5. Сопровождение

Команда `positioning track-demo` имитирует поток окон, применяет сглаживание и пишет CSV-журнал. В журнале видны статусы, углы, доверие, скачки, невязки и TDOA-пары.

6. ПМИ

Команда `positioning pmi` запускает набор проверок и формирует:

- ПРОТОКОЛ_ПМИ.md;
- `pmi_results.json`;
- `pmi_results.csv`.

Последний прогон закрыл 15 из 15 проверок.

Статусы результата

`LocalizationResult.status` принимает диагностические значения:

- `ok` - результат рассчитан и проходит проверки;
- `low_confidence` - решение найдено, но качество недостаточно высокое;
- `invalid_input` - входные данные некорректны;
- `insufficient_signal` - сигнал или окно недостаточно информативны.

Статус не заменяет численные метрики. Для анализа нужно смотреть также `confidence`, `residual_error2`, `used_pairs`, `rejected_pairs` и `reliable`.

Что считается готовым результатом практической части

Готовым результатом текущей версии является воспроизводимый стенд, который:

- запускается одной командой;
- использует JSON-конфигурацию;
- принимает или генерирует четыре канала;
- сохраняет метаданные;
- выполняет предобработку;
- проверяет TDOA на физическую согласованность;
- устойчив к единичному ложному пику;
- выдает направление/координаты в допустимых режимах;
- сглаживает последовательность оценок;
- формирует JSON/CSV-логи;
- автоматически закрывает таблицу ПМИ.

4.1. GCC-PHAT из аудио

Команды `positioning localization` и `positioning localize-audio` читают CSV/WAV, делят сигнал на окна, выполняют предобработку для диагностики информативности, оценивают TDOA через GCC-PHAT, передают задержки в локализатор и пишут CSV/JSON. `localization` дополнительно удобно использовать для демонстрации визуализации:

```
positioning localization --input .\data\synthetic\signal.csv --mode direction --show-visualization
```

Для каждой пары микрофонов сохраняются задержка, качество пика и вес. Далее эти пары проходят существующие проверки физической допустимости, транзитивности и робастного согласования.

Что еще требует проверки на реальном макете

Алгоритмический блок GCC-PHAT реализован, но для реальной аппаратной демонстрации нужны практические проверки:

- синхронность четырех каналов записи;
- отсутствие неизвестных аппаратных задержек между каналами;
- ориентация направленных микрофонов относительно рабочей полусферы;
- подбор частотной полосы под звук БПЛА;
- устойчивость в помещении с отражениями;
- достаточное отношение сигнал/шум.